

Testing and Debugging

Alexis Shakas Ian Wasser (TA)

ETH Zurich

2026-05-05

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	ML H41.1
6: Git collaboration	24/3	N	RZ D8
7: AI-assisted Python	31/3	Y	ML H41.1
8: The 3 R's in Python	14/4	Y	ML H41.1
9: The Python ecosystem	21/4	Y	RZ D8
10: cont. L9 & project feedback	28/4	N	RZ D8
11: Testing and debugging	5/5	Y	ML H41.1
12: OOP	12/5	Y	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A.: Take home assignment and peer review **today**.

Plan for today

Today is structured differently from previous lectures:

1. **Until 14h30** non-stop lecture
2. **10 min** break (Alexis runs to the train station)
3. **20 min** team game

Today's learning objectives

By the end of today, you should be able to:

- Debug programs using **traceback** and **breakpoints**
- Run pdb commands to **inspect a running program**
- Write a simple pytest test for a function using **assert**
- Explain what **Test Driven Development** is and how it helps you stay in the feedback loop with AI
- Ponder on the **AI-era trap**: when AI writes both the code and the tests, what are you actually verifying?

Debugging

Tracebacks: recap from Lecture 7

A **traceback** tells you **what** went wrong and **where**. Read it **bottom-up**: the last line is the error; the lines above are the call stack.

```
Traceback (most recent call last):  
  File "main.py", line 12, in <module>  
    result = average(numbers)  
  File "main.py", line 4, in average  
    return sum(numbers) / len(numbers)  
ZeroDivisionError: division by zero
```

But what if the program **doesn't crash** — it just gives the wrong answer? No traceback. We need to **pause and look inside**.

Our running example: count_words

```
def count_words(text: str) -> int:
    """Count whitespace-separated words in `text`.

A token counts as a word if it has any non-punctuation
character after stripping `skipchars` from both ends.
"""
    skipchars: str = ". , ! ? ; : \ ' ( ) [ ] "          # outer scope

    def is_word(token: str) -> bool:                    # inner function
        cleaned = token.strip(skipchars)                # reads outer scope
        return len(cleaned) > 0

    word_count = 0
    for token in text.split():                          # split() defaults to any whitespace
        if is_word(token):
            word_count += 1
    return word_count


```

We intentionally use a function within a function to demonstrate scope (LEGB, Lecture 8). You will find this function on Moodle.

A first breakpoint

Add `breakpoint()` on any line. By default, Python pauses there and hands you an interactive prompt.

`breakpoint()` is built in from Python 3.7 onward.

```
def count_words(text: str) -> int:
    breakpoint()                                # ← pause here
    skipchars = ". , ! ? ; : \ ' ( ) [ ] "
    ...

count_words("Alice said—oh dear!")
```


The pdb prompt

When you run the above code, python pauses at `breakpoint()` and gives you a (Pdb) prompt.

```
> count_words(text)
-> skipchars = ".,!?:\\"'()[ ]"
(Pdb)
```

Here are some of the most important commands to know at the prompt:

p	name	print	the value of name
n		next	line (run current line, stay in this function)
s		step	into the function being called on this line
c		continue	until the next breakpoint or the end
q		quit	the debugger

Inspecting scope at a breakpoint

Beyond `p`, `pdb` gives you several ways to ask “what’s visible here?”:

<code>p <name></code>	print the value of <code><name></code>
<code>pp <name></code>	pretty-print — better for dicts and lists
<code>locals()</code>	all local names as a dict (Local in LEGB)
<code>globals()</code>	all global names as a dict (Global in LEGB)
<code>a / args</code>	the arguments of the current function
<code>dir()</code>	every name visible in the current scope



Note

These are the same Python builtins you already know — `pdb` just runs them at the paused frame.

n vs s, and how scope shows up

Paused at `if is_word(token):` for `text = "Alice said—oh dear!"`:

n (next):

```
(Pdb) n  
> count_words(text)  
-> word_count += 1
```

Ran `is_word(token)`, stopped on the next line of `count_words`.

s (step):

```
(Pdb) s  
--Call--  
-> def is_word(token):  
(Pdb) p token  
'said—oh'  
(Pdb) p skipchars  
".,!?;:\\"'()[]"
```

Inside `is_word`. `p skipchars` reads the **enclosing scope**; LEGB live.

MiniTask: Use `pdb` to investigate `count_words`

Run `count_words("Alice said—oh dear!")` and figure out which tokens it counted.

1. Add `breakpoint()` at the start of `count_words`.
2. Use `n` until you reach `if is_word(token):`, then press `s` to step **into** `is_word`.
3. At each call, run `p token` and `p cleaned`. Note every token and whether it counted as a word.
4. What did `count_words` return? Why?

Same idea in your editor

The interface varies but the concept remains: **pause, inspect, step**.

- **VS Code:** click the gutter to set a breakpoint, hit **Run and Debug**.
- **PyCharm / Spyder:** GUI breakpoints + step buttons.
- **Jupyter:** `%debug` drops you into a pdb prompt.
- **ipdb:** prettier pdb (`pip install ipdb`).

The “**Variables**” panel in any of these is a **live picture of LEGB** and the buttons map one-to-one onto `n`, `s`, `c`.

If you **understand** the pdb commands, you can drive any of them.

Why test?

The bug we just found

Why test?

`count_words` didn't crash; it **quietly returned the wrong number**. We only found the bug because we went in with `pdb` and looked. The problem was with `skipchars`. It didn't include the em-dash (—), so `is_word` thought **said—oh** was one word, not two.

The bug we just found

Why test?

count_words didn't crash; it **quietly returned the wrong number**. We only found the bug because we went in with pdb and looked. The problem was with skipchars. It didn't include the em-dash (—), so is_word thought **said—oh** was one word, not two.

Question: if we patch skipchars, how do we know we didn't break the simple cases? How will **you** remember the em-dash catch six months from now?

The bug we just found

Why test?

`count_words` didn't crash; it **quietly returned the wrong number**. We only found the bug because we went in with `pdb` and looked. The problem was with `skipchars`. It didn't include the em-dash (—), so `is_word` thought **said—oh** was one word, not two.

Question: if we patch `skipchars`, how do we know we didn't break the simple cases? How will **you** remember the em-dash catch six months from now?

Answer: we write **tests**; small programs that check that `count_words` does what we expect on every case we care about. We run them **every time** we change the code, to make sure we didn't break anything.

A test is three things

A test is

1. an **input** you control
2. the **code** run on that input
3. an **assertion** about the (expected) output

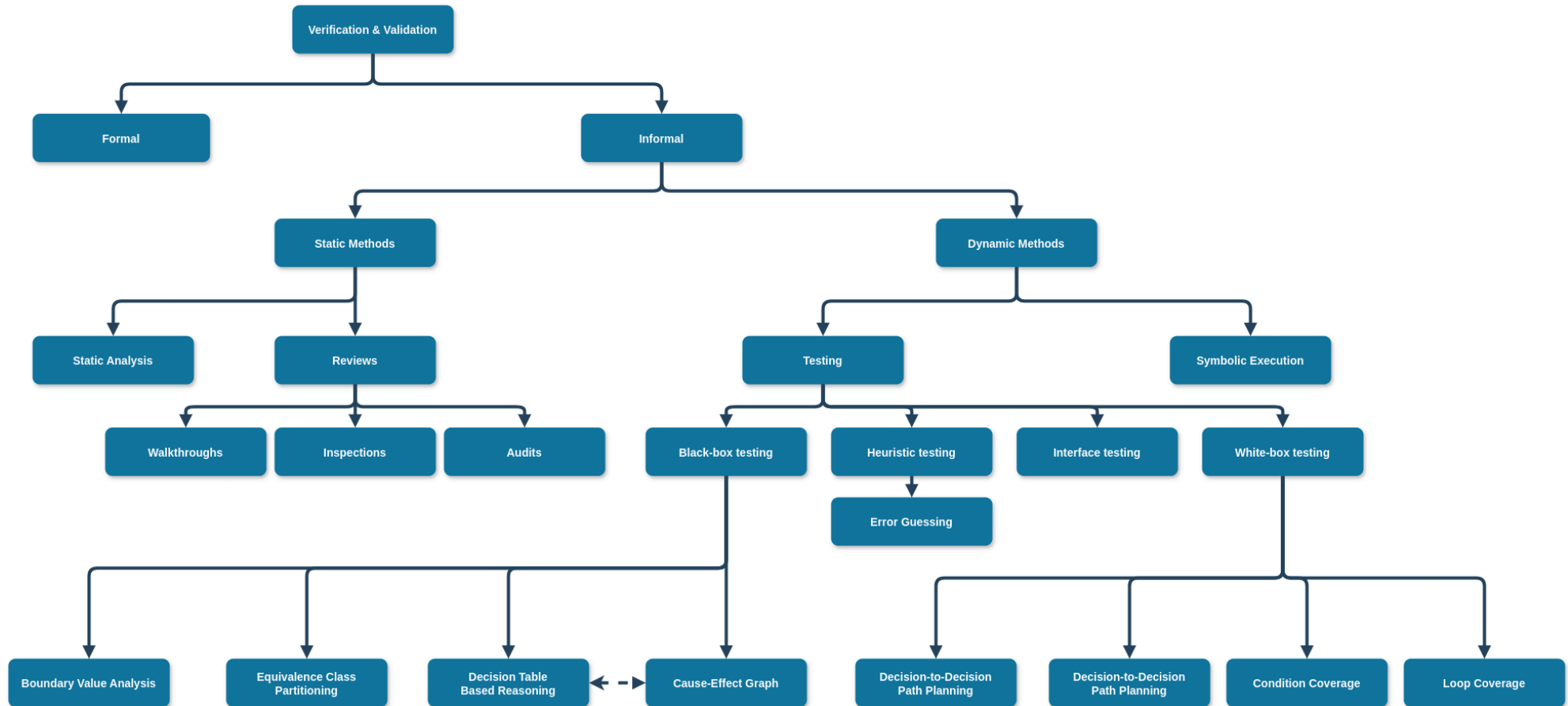
```
def test_simple():  
    result = count_words("Alice  
said hello")  
    assert result == 3
```

We'll cover assert in the next section.

Tests:

- **Catch bugs early** — ideally before users.
- **Allow for refactors** — change code freely, tests tell if you broke something.
- **Are documentation** — `count_words("said—oh") == 2` **is** the spec for em-dashes.

Types of testing



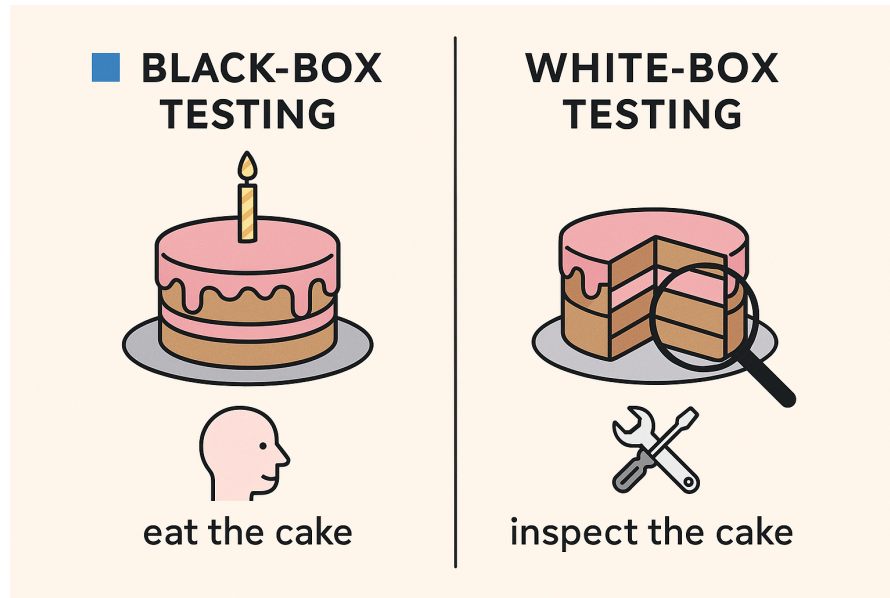
Today's focus: **dynamic, black-box, unit testing** Types of testing

Testing is one branch of **Verification & Validation**.

Today we focus on:

- **Dynamic** (run the code)
- **Black-box** (input $\rightarrow$ output)
- **Unit** level (one function)

Black-box vs white-box

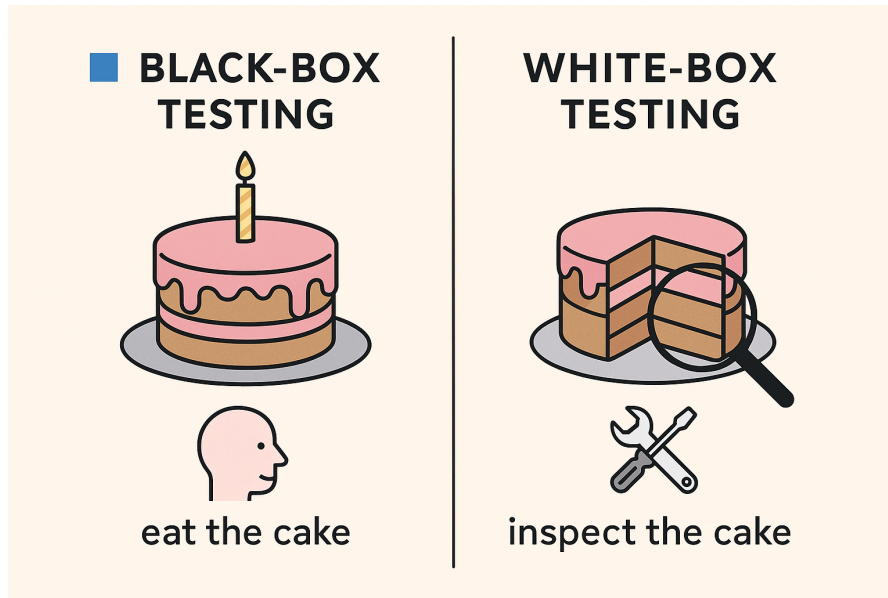


Will this cake kill you? You want to test it. How?

Black-box: you taste the cake. You don't know the recipe. You focus on the input → output relationship.

White-box: you inspect the recipe and look at the internal structure rather than output

Black-box vs white-box



Will this cake kill you? You want to test it. How?

Black-box: you taste the cake. You don't know the recipe. You focus on the input $\rightarrow$ output relationship.

White-box: you inspect the recipe and look at the internal structure rather than output

A test like `count_words("hi there") == 2` is **black-box** — we don't care **how** it counts, only that the result is right. All of today's tests (and the game) are black-box.

Testing in Python

assert: the building block, and two libraries

Testing in Python

assert is built-in:

```
>>> assert 1 + 1 == 2
# silent
>>> assert 1 + 1 == 3, "broken"
AssertionError: broken
```

Every test framework is, underneath, a fancier assert runner with a nice report.

unittest (built-in, class-based):

```
class TestCW(unittest.TestCase):
    def test_simple(self):
        self.assertEqual(
            count_words("hi there"), 2)
```

pytest (friendlier, better output):

```
def test_simple():
    assert count_words("hi there") == 2
```

assert is built-in:

```
>>> assert 1 + 1 == 2
# silent
>>> assert 1 + 1 == 3, "broken"
AssertionError: broken
```

Every test framework is, underneath, a fancier assert runner with a nice report.

pytest needs to be installed (`pip install pytest`) and is the suggested framework for this course.

unittest (built-in, class-based):

```
class TestCW(unittest.TestCase):
    def test_simple(self):
        self.assertEqual(
            count_words("hi there"), 2)
```

pytest (friendlier, better output):

```
def test_simple():
    assert count_words("hi there") == 2
```

MiniTask: Warm up with plain assert

Inside `analyse_text.py`, add **three** assert statements. Include:

1. One that should pass for a simple case, e.g. "hello world".
2. One edge case with punctuation at both ends, e.g.
"...hello!!!".
3. One that should fail on purpose so you can read the `AssertionError` message.

You can add these asserts inside the just below the function definition. Run the file to see the results.

Three rules:

1. File name starts with `test_` (e.g. `test_text_analysis.py`) and may contain multiple test functions.
2. Test function name starts with `test_` (e.g. `test_em_dash`).
3. Use plain `assert`.

```
from my_package import count_words

def test_simple():
    assert count_words("Alice said hello") == 3

def test_em_dash():
    assert count_words("Alice said—oh dear") == 4
```

Reading pytest output

In the terminal, run with `pytest` (or `pytest -v` for verbose).

```
===== test session starts =====
collected 2 items

test_count_words.py .F                                [100%]

===== FAILURES =====
_____ test_em_dash _____

    def test_em_dash():
>         assert count_words("Alice said-oh dear") == 4
E         assert 3 == 4
E         + where 3 = count_words('Alice said-oh dear')

test_count_words.py:5: AssertionError
===== 1 failed, 1 passed in 0.02s =====
```

MiniTask: Catch the em-dash bug, then fix it

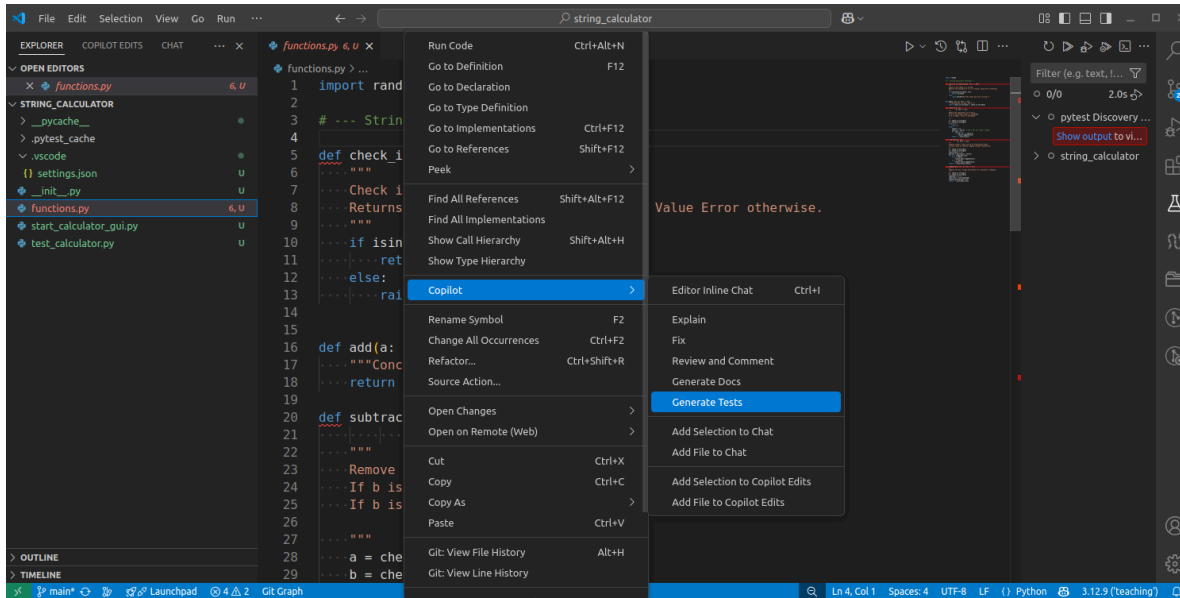
1. Install pytest if you haven't already (`pip install pytest`).
2. Create a `test_count_words.py` file.
3. Add **three** tests:
 - A simple sentence with no punctuation
 - A sentence that **uses an em-dash** (the bug we found)
 - One you invented — something a user might actually type
4. Run `pytest -v`. The em-dash test should **fail**.
5. **Fix** `count_words` so all three tests pass.
6. Re-run `pytest`. All green?

Testing in the era of AI

The shift

Testing in the era of AI

Writing tests used to be slow. So most people skipped them. Now AI writes code **and tests** in seconds.



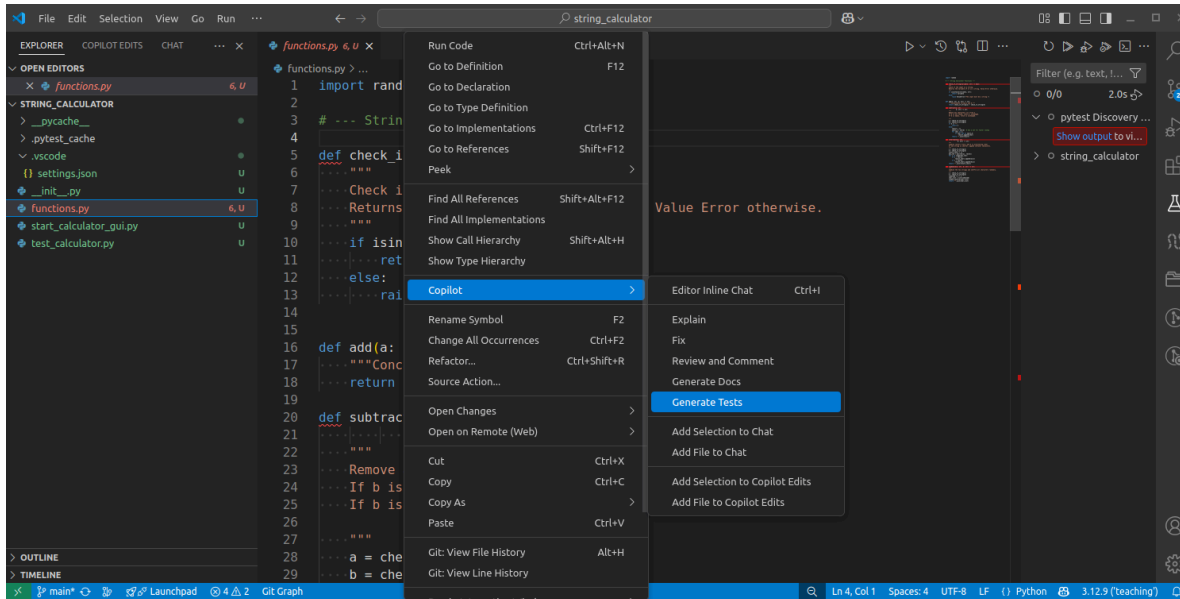
It can be as easy as...
right-click a function in
your editor →
“Generate tests”. Done.

So the question becomes:

The shift

Testing in the era of AI

Writing tests used to be slow. So most people skipped them. Now AI writes code **and tests** in seconds.



It can be as easy as...
right-click a function in
your editor →
“Generate tests”. Done.

So the question becomes:

If AI writes both the code **AND** the tests, what are you
actually testing?

Ask AI for `count_words`. Ask AI for tests. All tests pass  — yet the em-dash bug we found is **right there**.

```
def count_words(text):  
    skip = ". , ! ? ; : \ \"' () []"  
    return sum(  
        1 for t in text.split()  
        if t.strip(skip)  
    )
```

```
def test_simple():  
    assert count_words("hi there")  
    == 2  
  
def test_punctuation():  
    assert count_words("Hello,  
world!") == 2
```

The tests reflect the **implementation**, not your **intent**. If AI was never told that em-dashes mattered, then this bug will pass silently..until things break!

A discipline from the early 2000s — flip the usual order:

1. **Red** Write a **failing** test for the next bit of behavior.
2. **Green** Write the **minimum code** that makes it pass.
3. **Refactor** Clean up. Tests stay green. Repeat.

The output is **code plus a growing safety net of tests** that documents every decision you made along the way.

A discipline from the early 2000s — flip the usual order:

1. **Red** Write a **failing** test for the next bit of behavior.
2. **Green** Write the **minimum code** that makes it pass.
3. **Refactor** Clean up. Tests stay green. Repeat.

The output is **code plus a growing safety net of tests** that documents every decision you made along the way.

TDD predates AI by twenty years. It worked because tests forced you to **think before writing** — exactly what AI lets you skip if you're not careful.

The **spec** is now the artifact you own and AI writes the code:

1. **You** write the spec — as tests, examples, types, docstrings — **what** should this do, in every case that matters?
2. **AI** writes the implementation.
3. **Tests** tell you whether AI got it right. If not, sharpen the spec and try again.

Tests are how the human stays in the feedback loop.

The bottleneck is no longer writing code; it's **clarity of intent**. Code review becomes essential, with tests as the spec. The better your tests, the better your AI output.

Wrap-up

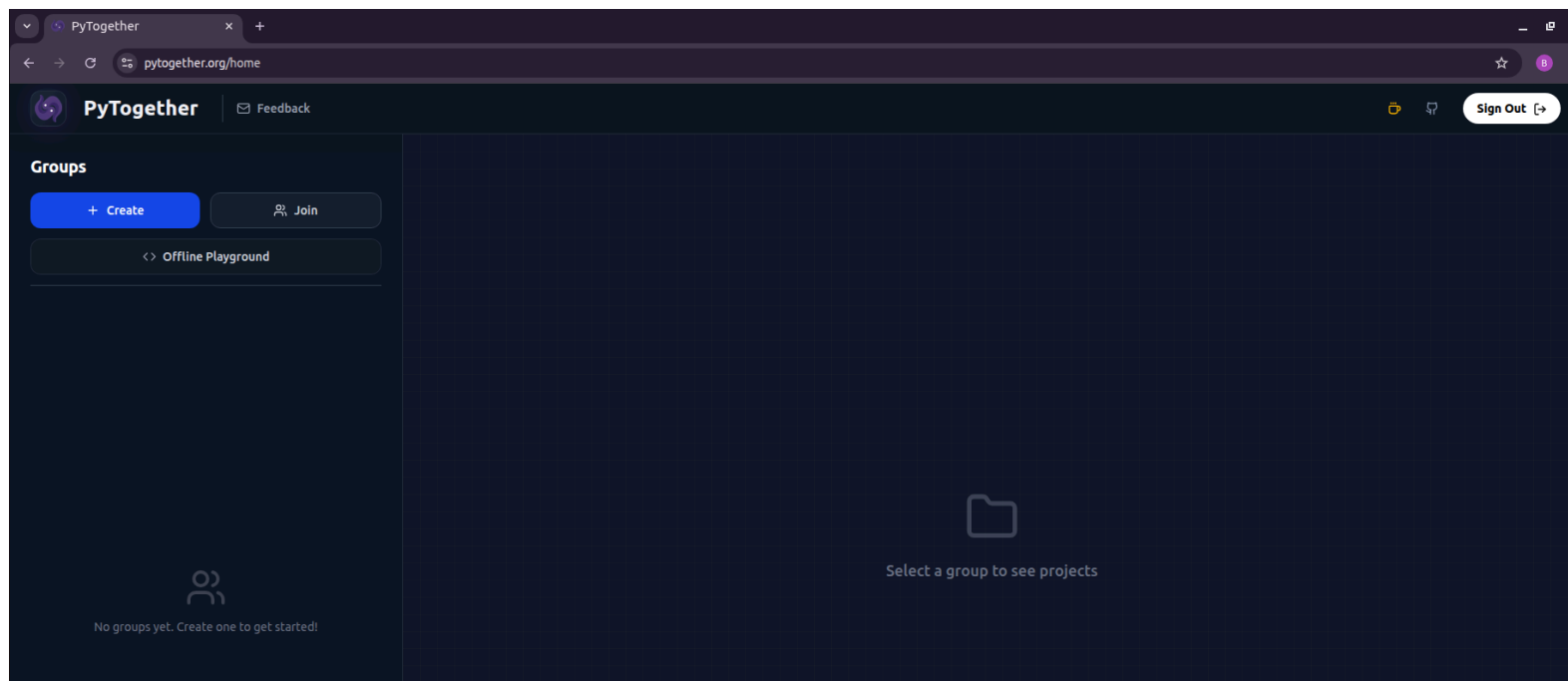
Today we covered:

- How to use `breakpoint()` and `pdb` to inspect a running program.
- How to leverage debugging to understand and fix bugs
- How to write a pytest test: file `test_*.py`, function `test_*`, plain `assert`.
- What is the **AI trap**: AI-generated tests reflect the implementation, not your intent — **you** are the one writing the spec.

Now it's time for a  **10-minute break**

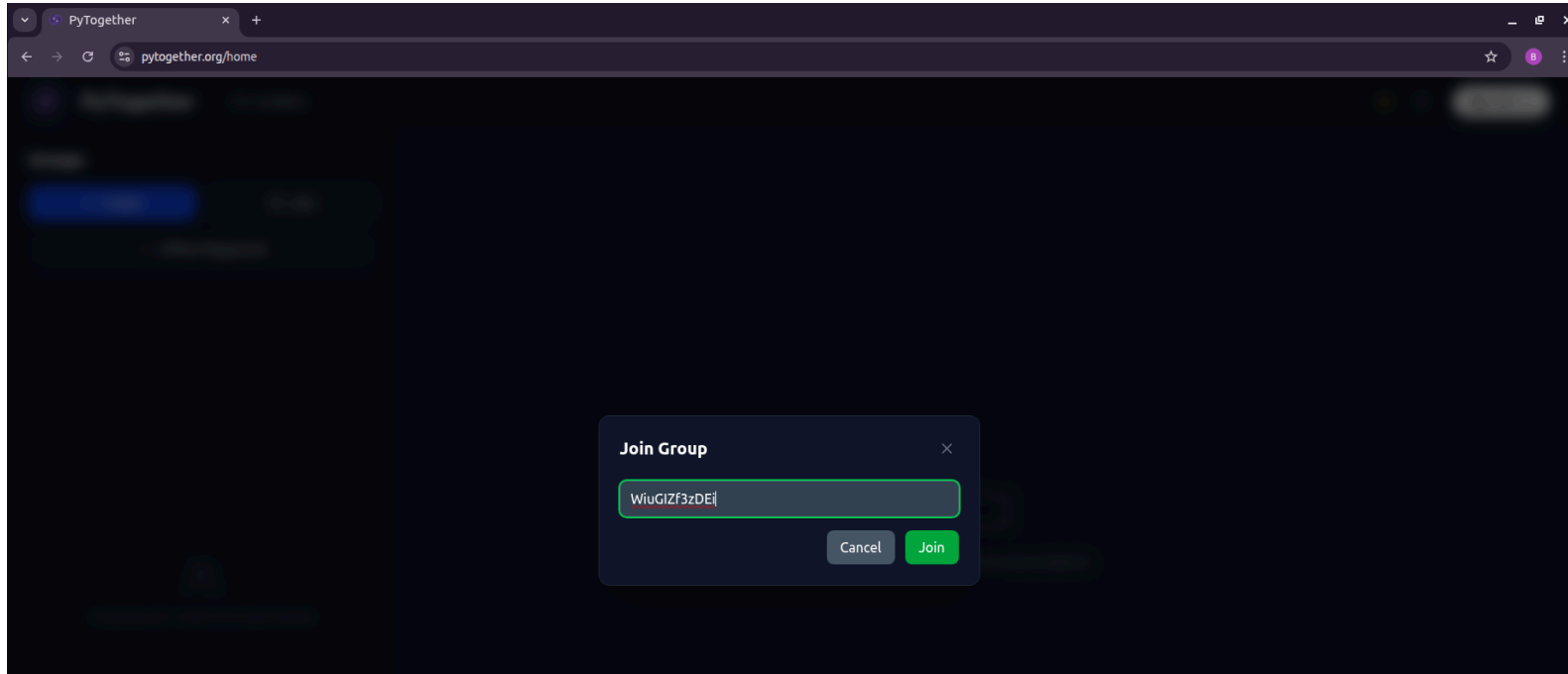
But before the break, let's quickly go over the game we'll play after the break.

Go to **pytogether.org**. Click **Join** and enter the group code I shared.



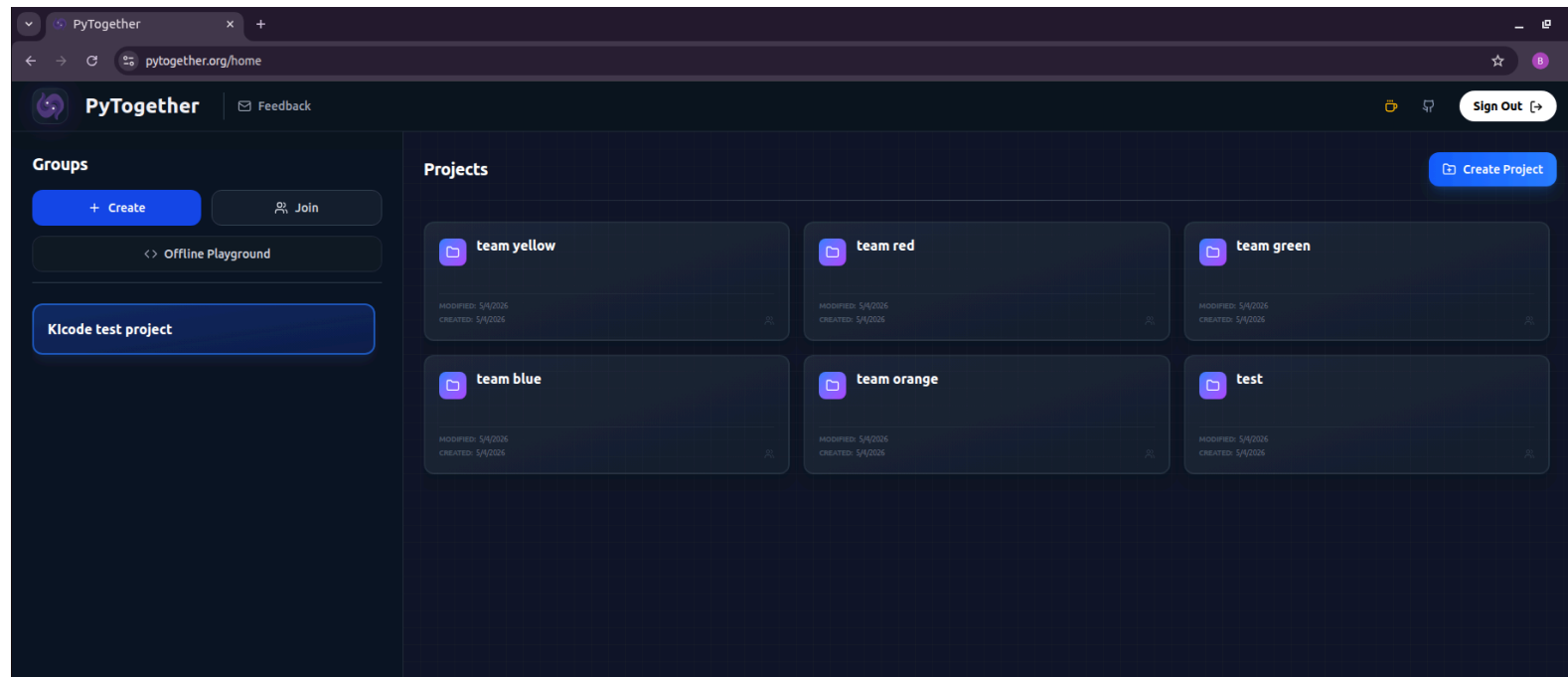
Step 1 (continued) — Enter the group code

Paste the code, then click **Join**.



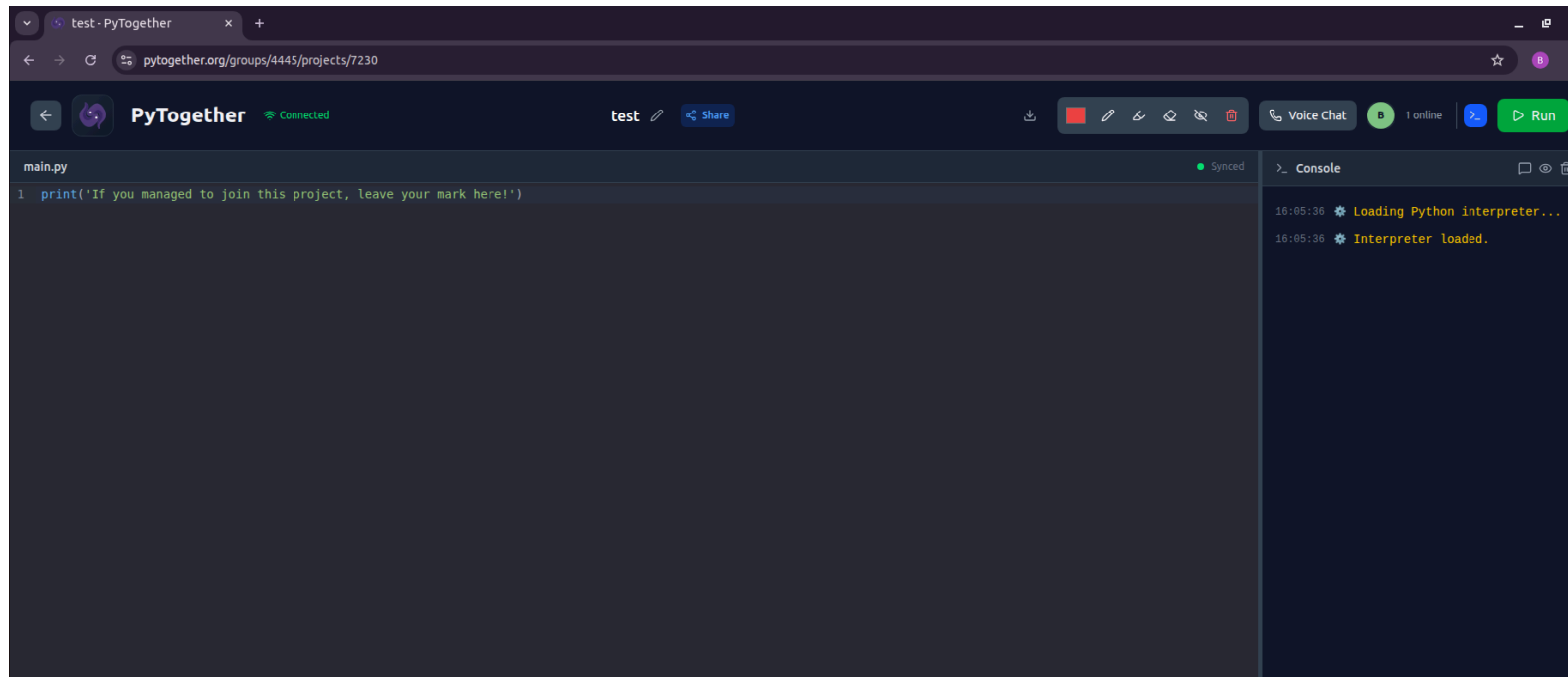
Step 2 — Pick your team's project

You'll see the **KIcode test project** group on the left. Each team has its own pre-made project (yellow, red, green, blue, orange). **Open the one for your team.**



Step 3 — Code together, in real time

Inside the project: a shared editor and a live console. Everyone in the team sees the cursor, the code, and the output as it changes. Hit **Run** to execute.



Format: Code vs Test groups

We'll play **spec-then-build**:

1. Each team splits into a **code** and a **test** group (2-3 people per group).
2. Both groups get the **same written spec** for a small text-processing function (not `count_words`).
3. They work **independently** for **10 minutes**. Both are allowed to use AI (outside of **pytogether**).
 - **code** group: write the functions.
 - **test** group: write pytest tests.
4. Reveal: Run tests against functions; how many fail?

- Within your group, you can
 1. either work simultaneously in one **pytogether** project (the one with your Team's color and group) or
 2. work together on one laptop.
- Code and tests should be in **one file each**: `solution.py` and `test_solution.py`.
- At the end of 10 min, download both scripts in one folder (`test_solution.py` and `solution.py`) and run `pytest`.

Rules of the game

- Try to do this **without AI** as much as possible.
- **No communication** between **code** and **test** groups during the 10 minutes.
- **Code** team should make sure their solution **matches the spec** and is robust to edge cases.
- Tests must be **runnable as-is** — if a test crashes (not just fails), it doesn't count.
- Test inputs must be **reasonable** and **reflect the spec**.

Write a function with this signature and docstring:

```
def count_sentences(text: str) -> int:  
    """Return the number of sentences in `text`."""
```

Test it on `alice.txt` — a **503-word excerpt** from Chapter I of **Alice in Wonderland**. The file is on moodle. You can read it in your code with:

```
with open("alice.txt") as f:  
    text = f.read()
```

The spec is **intentionally minimal**. **What is a sentence?** You decide — and your decision becomes the test.

Timer starts when teams are formed.

Alice was beginning to get very tired of sitting by her sister on the bank, and of having nothing to do: once or twice she had peeped into the book her sister was reading, but it had no pictures or conversations in it, “and what is the use of a book,” thought Alice “without pictures or conversations?”

So she was considering in her own mind (as well as she could, for the hot day made her feel very sleepy and stupid), whether the pleasure of making a daisy-chain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit with pink eyes ran close by her.

There was nothing so *very* remarkable in that; nor did Alice think it so *very* much out of the way to hear the Rabbit say to itself, “Oh dear! Oh dear! I shall be late!” (when she thought it over afterwards, it occurred to her that she ought to have wondered at this, but at the time it all seemed quite natural); but when the Rabbit actually *took a watch out of its waistcoat-pocket*, and looked at it, and then hurried on, Alice started to her feet, for it flashed across her mind that she had never before seen a rabbit with either a waistcoat-pocket, or a watch to take out of it, and burning with curiosity, she ran across the field after it, and fortunately was just in time to see it pop down a large rabbit-hole under the hedge.

In another moment down went Alice after it, never once considering how in the world she was to get out again.

The rabbit-hole went straight on like a tunnel for some way, and then dipped suddenly down, so suddenly that Alice had not a moment to think about stopping herself before she found herself falling down a very deep well.

Either the well was very deep, or she fell very slowly, for she had plenty of time as she went down to look about her and to wonder what was going to happen next. First, she tried to look down and make out what she was coming to, but it was too dark to see anything; then she looked at the sides of the well, and noticed that they were filled with cupboards and book-shelves; here and there she saw maps and pictures hung upon pegs. She took down a jar from one of the shelves as she passed; it was labelled “ORANGE MARMALADE”, but to her great disappointment it was empty: she did not like to drop the jar for fear of killing somebody underneath, so managed to put it into one of the cupboards as she fell past it.

“Well!” thought Alice to herself, “after such a fall as this, I shall think nothing of tumbling down stairs! How brave they’ll all think me at home! Why, I wouldn’t say anything about it, even if I fell off the top of the house!” (Which was very likely true.)